

# PROBA: Provenance-Based Bias Auditor Guide

This document provides clear, end-to-end instructions for installing dependencies, running the application, and conducting bias audits using PROBA. This tool is designed to identify fairness across sequential data preprocessing pipelines through stochastic metaheuristic search (Hybrid WOA-DE).

## 1. Initial Setup

If you have newly cloned or downloaded the project repository, you must install the required dependencies for both the Backend and Frontend.

### A. Backend Setup

Open your terminal or command prompt in the project root directory. Install the required Python packages by executing:

```
pip install fastapi "uvicorn[standard]" websockets python-multipart pydantic pandas  
numpy psycpg2-binary psutil python-dotenv folktables ipykernel
```

### B. Frontend Setup

Open a terminal, navigate into the frontend directory, and install all JavaScript/TypeScript dependencies:

```
cd frontend  
npm install
```

## 2. Running the Application

You can run the whole system by simply running the "run.py".

### Step A: Run "run.py"

Confirm that your terminal displays the local server URL as shown below (typically <http://localhost:5173/>):



Figure 3: <http://localhost:5173/> on Terminal

### Step B: Access the PROBA Dashboard

Open your web browser and navigate to: <http://localhost:5173/dashboard>

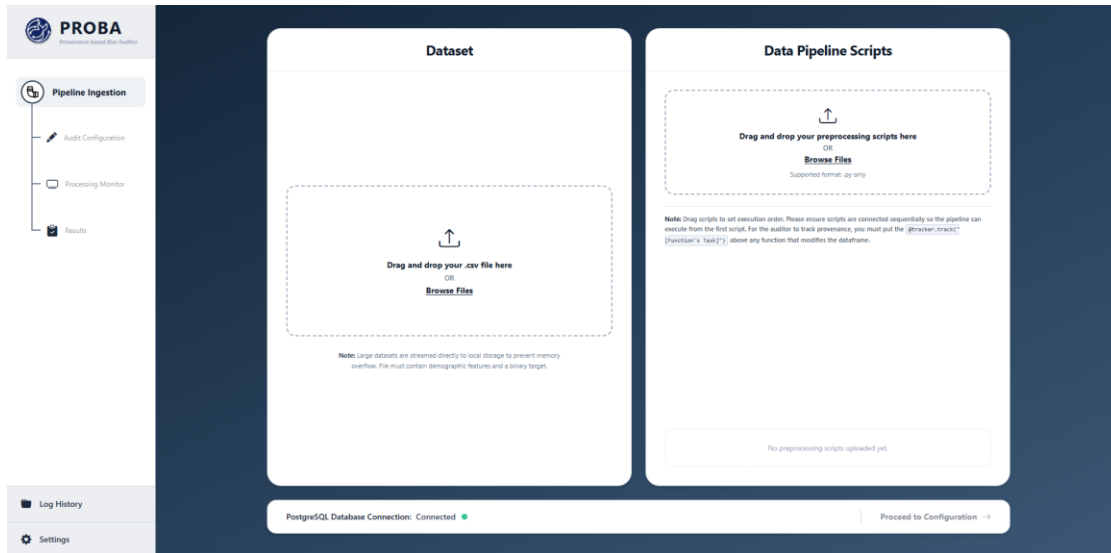


Figure 5: PROBA Dashboard

### 3. Step-by-Step Audit

Follow this six-step walkthrough to perform a complete bias audit on a sample data preprocessing pipeline.

#### Step 1: Download Dataset & Preprocessing Scripts

To test the bias detector, download the sample dataset and sequential preprocessing scripts from the drive folder: [Inputs For The Tool's UI \(Google Drive\)](#)

#### Step 2: Upload Dataset and Arrange Preprocessing Pipeline

On the Dashboard page, drag and drop or upload the downloaded dataset and all preprocessing scripts into their respective dropzones.

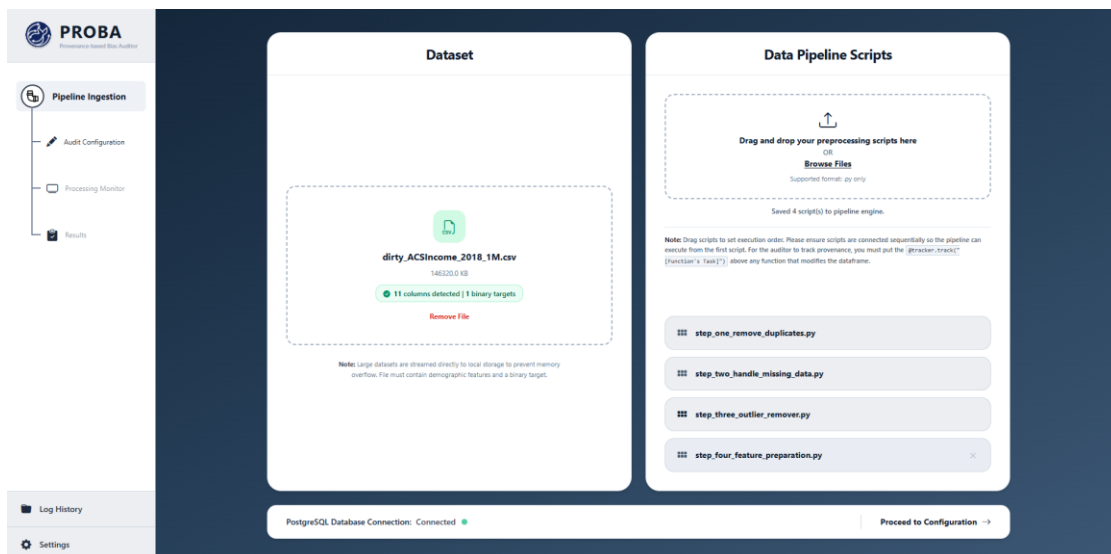


Figure 6: Uploading dataset and arranging preprocessing pipeline scripts

**CRITICAL:** Ensure that your preprocessing scripts are sorted in sequential execution order matching their intended workflow. PROBA traces pipeline transformations sequentially to detect bias emergence at each stage.

Once verified, click the Proceed to Configuration button.

### Step 3: Configure

In the Configuration panel, configure the audit parameters as shown below:

The screenshot displays the 'Audit Configuration Panel' with two main sections:

- Select and Configure Protected Demographic Attributes:** This section features a dropdown menu labeled 'Select column/s from dataset'. Below it, a 'Column/s Selected' box lists three attributes: 'age', 'sex', and 'race'. Each attribute has a 'Categorical' button (highlighted in blue) and a 'Continuous' button.
- Define the Binary Target Variable and its Outcome Values:** This section includes three dropdown menus: 'Select binary target column from the dataset' (set to 'Income'), 'Select favorable outcome value' (set to 'True'), and 'Select unfavorable outcome value' (set to 'False').

At the bottom of each section, there is a small informational icon and text:

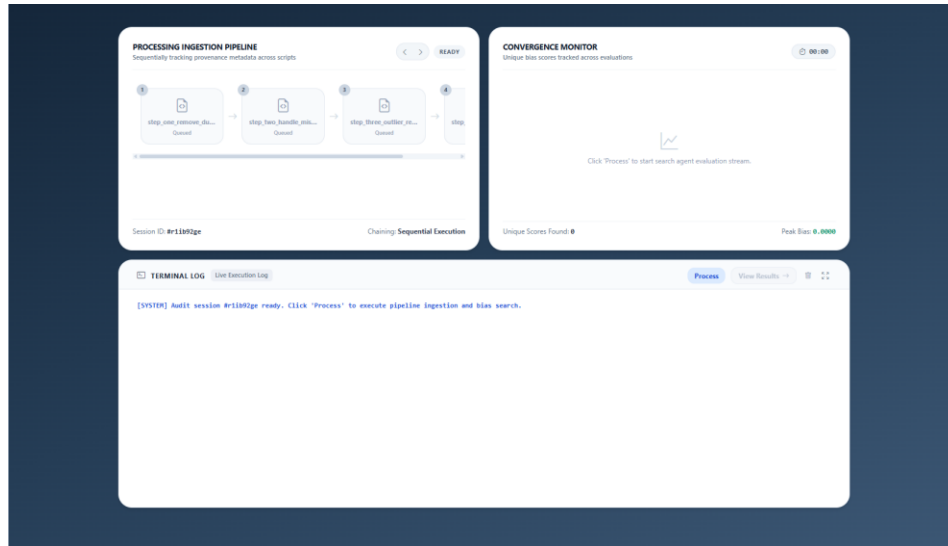
- For the first section: 'This establishes the core demographic variables that the system will monitor and allows the system to analyze combinations of characteristics to expose the highest bias.'
- For the second section: 'Defining outcome values is required to compute selection rates for mathematical bias calculation.'

Figure 7: Audit Configuration Panel

Click Next to proceed to the live Processing page.

### Step 4: Execute

On the Processing page, locate the Terminal Log card. Click the Process button in the terminal header to start execution.

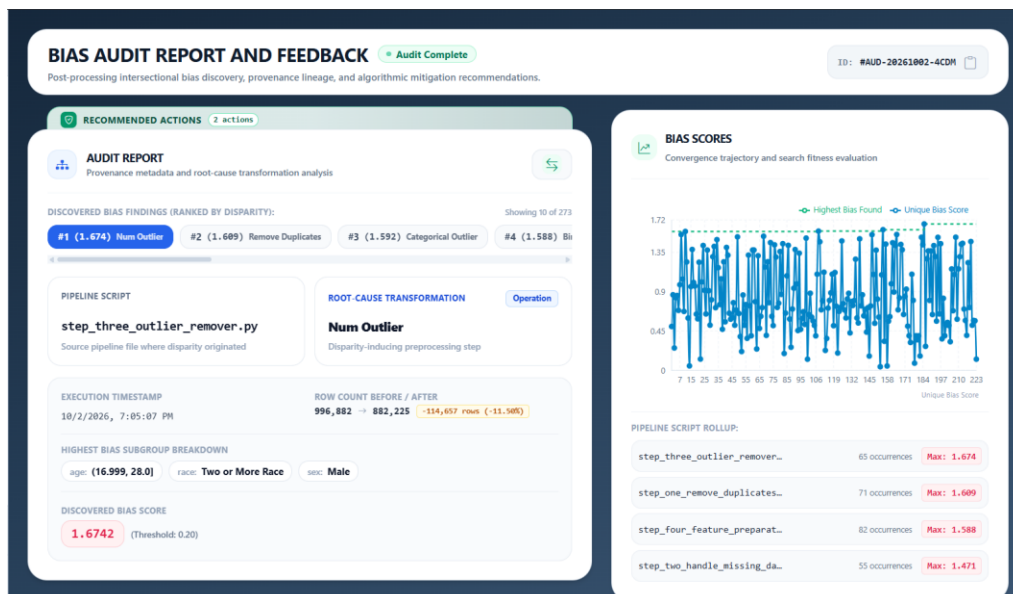


**EXECUTION DURATION:** Processing time depends on your computer's CPU specifications and dataset size. For large benchmark datasets (e.g., ~1 million rows), the audit may take several minutes.

Once the audit completes, the View Results button will become active and turn green. Click View Results to inspect the comprehensive findings.

## Step 5: Bias Findings & Mitigation Strategies

The Results page displays the full Bias Audit Report, highlighting the highest bias score discovered across the pipeline transformations:



- **Benchmark Score:** A bias score of approximately 1.4922 represents the benchmark maximum bias detected on the reference test dataset. If your run yields a slightly different score, this is expected due to the stochastic search exploration of the Hybrid WOA-DE metaheuristic algorithm.
- **Ranked Findings:** The Ranked Discovered Bias Findings table lists the top 10 most severe subgroup disparities. Click 'See More' to view additional findings in increments of 10.
- **Mitigation Strategies:** Dedicated Recommended Actions (Mitigation Strategy) section for specific data preprocessing remedies (such as targeted re-weighting or feature disentanglement).

**NOTE:** The goal of this tool is to find the highest bias in the user's inputs. In our case the highest bias is in:

**Script Name:** step\_three\_outlier\_remover.py

**Transformation Name:** Num Outlier

**Bias Score:** 1.67

#### Highest Bias In Each Transformation

- {'max\_fitness\_score': 1.6085982752251224, 'script\_name': 'step\_one\_remove\_duplicates.py', 'transformation\_name': 'Remove Duplicates', 'demographic\_group': 'age:{-5.001, 28.0]|race:Black|sex:M'}
- {'max\_fitness\_score': 1.4706890756518898, 'script\_name': 'step\_one\_remove\_duplicates.py', 'transformation\_name': 'Fix Format', 'demographic\_group': 'age:{-5.001, 28.0]|race:Other Race|sex:Male'}
- {'max\_fitness\_score': 1.4705597241630093, 'script\_name': 'step\_two\_handle\_missing\_data.py', 'transformation\_name': 'Handle Missing Rows', 'demographic\_group': 'age:{-5.001, 28.0]|race:Other Race|sex:Male'}
- {'max\_fitness\_score': 1.4705597241630093, 'script\_name': 'step\_two\_handle\_missing\_data.py', 'transformation\_name': 'Remove Missing Target Variable', 'demographic\_group': 'age:{-5.001, 28.0]|race:Other Race|sex:Male'}
- {'max\_fitness\_score': 1.674232980906194, 'script\_name': 'step\_three\_outlier\_remover.py', 'transformation\_name': 'Num Outlier', 'demographic\_group': 'age:{16.999, 28.0]|race:Two or More Race|sex:Male'}
- {'max\_fitness\_score': 1.6307724316959797, 'script\_name': 'step\_three\_outlier\_remover.py', 'transformation\_name': 'Categorical Outlier', 'demographic\_group': 'age:{16.999, 28.0]|race:Other Race|sex:Male'}
- {'max\_fitness\_score': 1.6307724316959797, 'script\_name': 'step\_four\_feature\_preparation.py', 'transformation\_name': 'Group Rare Classes', 'demographic\_group': 'age:{16.999, 28.0]|race:Other Race|sex:Male'}
- {'max\_fitness\_score': 1.5884136762299161, 'script\_name': 'step\_four\_feature\_preparation.py', 'transformation\_name': 'Bin Numerical Features', 'demographic\_group': 'age:Young Adult|race:Other Race|sex:Male'}
- {'max\_fitness\_score': 1.5884136762299161, 'script\_name': 'step\_four\_feature\_preparation.py', 'transformation\_name': 'Handle Missing Values', 'demographic\_group': 'age:Young Adult|race:Other Race|sex:Male'}

## Step 6: Reset Session

After completing an audit session, it is recommended to clear your temporary workspace data before auditing a new dataset or pipeline. Navigate to Settings in the sidebar and click Restart Session:

**SETTINGS** Restart Session

**DATABASE SETUP**

localhost

5432

bcr\_audit\_db

postgres

**WHALE OPTIMIZATION ALGORITHM**

Number of Searching Agents: 30

Maximum Number of Iterations: 500

**DIFFERENTIAL EVOLUTION**

Population Size: 50

Scale Factor: 0.5

Crossover Rate: 0.9

Maximum Stagnation Limit: 100

**BIAS THRESHOLD** 0.50

PostgreSQL Database Connection: Connected

Reset to Default Apply

Figure 10: Settings page

This clears the current session cache, frees server memory, and resets the application for a clean, reproducible run.